```
# Title Text
# Author Text
# Author Text
# Abstract Text
```

## Q1: Autoencoder Training for Dimensionality Reduction

- Loaded the county and state-level Trump-only compressed autoencoder to learn a low-dimensional representation of the gene expression data. ● Use two different latent sizes (code_size): 16 and 32 (i.e., mapping expressions to 16 dimensions and then from 16 back to 2000). ● Train the models for approximately 50 epochs (adjust as needed for optimal results). ● Record the training and validation losses.

```
# Clone the repo
!git clone https://github.com/ohsu-comp-bio/tcga-2018-gtex-v8.git
# Navigate to the data directory
cd tcga-2018-gtex-v8/data/
# Download the file using curl
!curl -L -o tcga_target_gtex_rsem_gene_tpm.gz https://xena.ucsc.edu/download/tcga-gtex-target-hub/tcga_target_gtex_rsem_gene_tpm.gz
# Unzip the file
!gunzip tcga_target_gtex_rsem_gene_tpm.gz
# Check the current directory and list files to see the downloaded file
!ls -l
```

```
Cloning into 'tcga-2018-gtex-v8'...
remote: Enter passphrase for key '/root/.ssh/id_rsa':
fatal: Authentication failed for 'https://github.com/ohsu-comp-bio/tcga-2018-gtex-v8.git'
/bin/bash: line 1: cd: tcga-2018-gtex-v8/data/: No such file or directory
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left  Speed
100 1.1G  100 1.1G    0     0  2.7M      0  0:07:09  0:07:09 --:--:-- 2.7M
gzip: tcga_target_gtex_rsem_gene_tpm.gz: unexpected end of file
total 1.1G
-rw-r--r-- 1 root root 1.1G Jun  1 00:00 tcga_target_gtex_rsem_gene_tpm.gz
```

- Loaded a fully connected autoencoder to learn a low-dimensional representation of the gene expression data.
- Use two different latent sizes (code_size): 16 and 32 (i.e., mapping expressions to 16 dimensions and then from 16 back to 2000).
- Train the models for approximately 50 epochs (adjust as needed for optimal results).
- Record the training and validation losses.

```
# Load the data
import pandas as pd
df = pd.read_csv("tcga_target_gtex_rsem_gene_tpm", sep="\t")
# Transpose the data so that samples are rows and genes are columns
df = df.transpose()
# Display the first few rows of the transposed dataset
df.head()
```

```
ERR315325,ERR315326,ERR315327,ERR315328,ERR315329,ERR315330,ERR315331,ERR315332,ERR315333,ERR315334,...
sample,_gene_rsem_id,_gene_chrom,_gene_start,_gene_end,_strand,_gene_type,_gene_name,_transcript_rsem_id,_transcript_chrom,_transcript_start,_transcript_end,_exon_count,_exon_starts,_exon_ends,_cds_start,_cds_end,_cds_exon_count,_exon_frames,_gene_aliases,_description,_hgnc_id,_ncbi_id,_gene_pattern,_transcript_pattern,_transcript_type,_transcript_name,_transcript_pattern,_transcript_type,_transcript_name,_transcript_pattern,_transcript_type,_transcript_name
```

```
Epoch 26/50
125/125 ━━━━━━━━━━━━━━━━━━━━ 1s 11ms/step - loss: 2.5422 - val_loss: 2.5579
Epoch 27/50
125/125 ━━━━━━━━━━━━━━━━━━━━ 1s 8ms/step - loss: 2.5468 - val_loss: 2.5558
Epoch 28/50
125/125 ━━━━━━━━━━━━━━━━━━━━ 1s 8ms/step - loss: 2.5402 - val_loss: 2.5565
Epoch 29/50
125/125 ━━━━━━━━━━━━━━━━━━━━ 1s 8ms/step - loss: 2.5343 - val_loss: 2.5556
Epoch 30/50
125/125 ━━━━━━━━━━━━━━━━━━━━ 1s 8ms/step - loss: 2.5343 - val_loss: 2.5550
Epoch 31/50
125/125 ━━━━━━━━━━━━━━━━━━━━ 1s 10ms/step - loss: 2.5395 - val_loss: 2.5536
Epoch 32/50
125/125 ━━━━━━━━━━━━━━━━━━━━ 1s 7ms/step - loss: 2.5358 - val_loss: 2.5554
Epoch 33/50
125/125 ━━━━━━━━━━━━━━━━━━━━ 1s 8ms/step - loss: 2.5335 - val_loss: 2.5547
Epoch 34/50
125/125 ━━━━━━━━━━━━━━━━━━━━ 2s 12ms/step - loss: 2.5348 - val_loss: 2.5535
Epoch 35/50
125/125 ━━━━━━━━━━━━━━━━━━━━ 2s 10ms/step - loss: 2.5383 - val_loss: 2.5560
Epoch 36/50
125/125 ━━━━━━━━━━━━━━━━━━━━ 2s 8ms/step - loss: 2.5356 - val_loss: 2.5565
Epoch 37/50
125/125 ━━━━━━━━━━━━━━━━━━━━ 1s 7ms/step - loss: 2.5311 - val_loss: 2.5546
Epoch 38/50
125/125 ━━━━━━━━━━━━━━━━━━━━ 1s 8ms/step - loss: 2.5296 - val_loss: 2.5542
Epoch 39/50
125/125 ━━━━━━━━━━━━━━━━━━━━ 1s 8ms/step - loss: 2.5389 - val_loss: 2.5545
Epoch 40/50
125/125 ━━━━━━━━━━━━━━━━━━━━ 1s 8ms/step - loss: 2.5391 - val_loss: 2.5539
Epoch 41/50
125/125 ━━━━━━━━━━━━━━━━━━━━ 1s 8ms/step - loss: 2.5305 - val_loss: 2.5537
Epoch 42/50
125/125 ━━━━━━━━━━━━━━━━━━━━ 1s 8ms/step - loss: 2.5403 - val_loss: 2.5545
Epoch 43/50
125/125 ━━━━━━━━━━━━━━━━━━━━ 1s 5ms/step - loss: 2.5308 - val_loss: 2.5532
Epoch 44/50
125/125 ━━━━━━━━━━━━━━━━━━━━ 1s 8ms/step - loss: 2.5298 - val_loss: 2.5563
Epoch 45/50
125/125 ━━━━━━━━━━━━━━━━━━━━ 1s 8ms/step - loss: 2.5353 - val_loss: 2.5525
Epoch 46/50
125/125 ━━━━━━━━━━━━━━━━━━━━ 1s 8ms/step - loss: 2.5381 - val_loss: 2.5549
Epoch 47/50
125/125 ━━━━━━━━━━━━━━━━━━━━ 1s 8ms/step - loss: 2.5312 - val_loss: 2.5531
Epoch 48/50
125/125 ━━━━━━━━━━━━━━━━━━━━ 1s 8ms/step - loss: 2.5265 - val_loss: 2.5611
Epoch 49/50
125/125 ━━━━━━━━━━━━━━━━━━━━ 1s 8ms/step - loss: 2.5395 - val_loss: 2.5524
Epoch 50/50
125/125 ━━━━━━━━━━━━━━━━━━━━ 1s 8ms/step - loss: 2.5283 - val_loss: 2.5547
157/157 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - loss: 2.5351
125/125 ━━━━━━━━━━━━━━━━━━━━ 2s 7ms/step - 7.2359 - loss: 2.6254
Epoch 1/50
125/125 ━━━━━━━━━━━━━━━━━━━━ 1s 8ms/step - loss: 2.7586 - val_loss: 2.6721
Epoch 2/50
125/125 ━━━━━━━━━━━━━━━━━━━━ 1s 7ms/step - loss: 2.6314 - val_loss: 2.5980
Epoch 3/50
125/125 ━━━━━━━━━━━━━━━━━━━━ 1s 8ms/step - loss: 2.5676 - val_loss: 2.5590
Epoch 4/50
125/125 ━━━━━━━━━━━━━━━━━━━━ 1s 5ms/step - loss: 2.5206 - val_loss: 2.5384
Epoch 5/50
125/125 ━━━━━━━━━━━━━━━━━━━━ 1s 5ms/step - loss: 2.4928 - val_loss: 2.5118
Epoch 6/50
125/125 ━━━━━━━━━━━━━━━━━━━━ 1s 5ms/step - loss: 2.4738 - val_loss: 2.4926
Epoch 7/50
125/125 ━━━━━━━━━━━━━━━━━━━━ 1s 6ms/step - loss: 2.4484 - val_loss: 2.4766
Epoch 8/50
125/125 ━━━━━━━━━━━━━━━━━━━━ 1s 5ms/step - loss: 2.4208 - val_loss: 2.4781
Epoch 9/50
125/125 ━━━━━━━━━━━━━━━━━━━━ 1s 5ms/step - loss: 2.4060 - val_loss: 2.4558
Epoch 10/50
125/125 ━━━━━━━━━━━━━━━━━━━━ 1s 5ms/step - loss: 2.3959 - val_loss: 2.4585
Epoch 11/50
125/125 ━━━━━━━━━━━━━━━━━━━━ 1s 5ms/step - loss: 2.3795 - val_loss: 2.4422
Epoch 12/50
125/125 ━━━━━━━━━━━━━━━━━━━━ 1s 5ms/step - loss: 2.3748 - val_loss: 2.4361
Epoch 13/50
125/125 ━━━━━━━━━━━━━━━━━━━━ 1s 5ms/step - loss: 2.3592 - val_loss: 2.4319
Epoch 14/50
125/125 ━━━━━━━━━━━━━━━━━━━━ 1s 5ms/step - loss: 2.3515 - val_loss: 2.4285
Epoch 15/50
125/125 ━━━━━━━━━━━━━━━━━━━━ 1s 5ms/step - loss: 2.3447 - val_loss: 2.4265
Epoch 16/50
125/125 ━━━━━━━━━━━━━━━━━━━━ 1s 5ms/step - loss: 2.3384 - val_loss: 2.4429
Epoch 17/50
125/125 ━━━━━━━━━━━━━━━━━━━━ 1s 4ms/step - loss: 2.3377 - val_loss: 2.4245
Epoch 18/50
125/125 ━━━━━━━━━━━━━━━━━━━━ 1s 5ms/step - loss: 2.3342 - val_loss: 2.4196
Epoch 19/50
125/125 ━━━━━━━━━━━━━━━━━━━━ 1s 5ms/step - loss: 2.3168 - val_loss: 2.4193
Epoch 20/50
125/125 ━━━━━━━━━━━━━━━━━━━━ 1s 5ms/step - loss: 2.3181 - val_loss: 2.4158
Epoch 21/50
125/125 ━━━━━━━━━━━━━━━━━━━━ 1s 6ms/step - loss: 2.3172 - val_loss: 2.4287
Epoch 22/50
125/125 ━━━━━━━━━━━━━━━━━━━━ 1s 6ms/step - loss: 2.3150 - val_loss: 2.4139
Epoch 23/50
125/125 ━━━━━━━━━━━━━━━━━━━━ 1s 5ms/step - loss: 2.3144 - val_loss: 2.4185
Epoch 24/50
125/125 ━━━━━━━━━━━━━━━━━━━━ 1s 7ms/step - loss: 2.3052 - val_loss: 2.4106
Epoch 25/50
125/125 ━━━━━━━━━━━━━━━━━━━━ 1s 7ms/step - loss: 2.2992 - val_loss: 2.4184
Epoch 26/50
125/125 ━━━━━━━━━━━━━━━━━━━━ 1s 7ms/step - loss: 2.3043 - val_loss: 2.4083
Epoch 27/50
125/125 ━━━━━━━━━━━━━━━━━━━━ 2s 10ms/step - loss: 2.2974 - val_loss: 2.4087
Epoch 28/50
125/125 ━━━━━━━━━━━━━━━━━━━━ 1s 8ms/step - loss: 2.3025 - val_loss: 2.4072
Epoch 29/50
125/125 ━━━━━━━━━━━━━━━━━━━━ 1s 8ms/step - loss: 2.2918 - val_loss: 2.4164
Epoch 30/50
125/125 ━━━━━━━━━━━━━━━━━━━━ 1s 8ms/step - loss: 2.2847 - val_loss: 2.4067
Epoch 31/50
125/125 ━━━━━━━━━━━━━━━━━━━━ 1s 8ms/step - loss: 2.2910 - val_loss: 2.4051
Epoch 32/50
125/125 ━━━━━━━━━━━━━━━━━━━━ 1s 8ms/step - loss: 2.2944 - val_loss: 2.4088
Epoch 33/50
125/125 ━━━━━━━━━━━━━━━━━━━━ 1s 6ms/step - loss: 2.2963 - val_loss: 2.4084
Epoch 34/50
125/125 ━━━━━━━━━━━━━━━━━━━━ 1s 8ms/step - loss: 2.2887 - val_loss: 2.4148
Epoch 35/50
125/125 ━━━━━━━━━━━━━━━━━━━━ 1s 8ms/step - loss: 2.2882 - val_loss: 2.4072
Epoch 36/50
125/125 ━━━━━━━━━━━━━━━━━━━━ 1s 8ms/step - loss: 2.2908 - val_loss: 2.4032
Epoch 37/50
125/125 ━━━━━━━━━━━━━━━━━━━━ 1s 8ms/step - loss: 2.2821 - val_loss: 2.4028
Epoch 38/50
125/125 ━━━━━━━━━━━━━━━━━━━━ 1s 6ms/step - loss: 2.2798 - val_loss: 2.4023
Epoch 39/50
125/125 ━━━━━━━━━━━━━━━━━━━━ 1s 8ms/step - loss: 2.2807 - val_loss: 2.4087
Epoch 40/50
125/125 ━━━━━━━━━━━━━━━━━━━━ 1s 8ms/step - loss: 2.2837 - val_loss: 2.4033
Epoch 41/50
125/125 ━━━━━━━━━━━━━━━━━━━━ 1s 6ms/step - loss: 2.2865 - val_loss: 2.4034
Epoch 42/50
125/125 ━━━━━━━━━━━━━━━━━━━━ 1s 8ms/step - loss: 2.2828 - val_loss: 2.4023
Epoch 43/50
125/125 ━━━━━━━━━━━━━━━━━━━━ 1s 8ms/step - loss: 2.2739 - val_loss: 2.4033
Epoch 44/50
125/125 ━━━━━━━━━━━━━━━━━━━━ 1s 8ms/step - loss: 2.2698 - val_loss: 2.4106
Epoch 45/50
125/125 ━━━━━━━━━━━━━━━━━━━━ 1s 7ms/step - loss: 2.2153 - val_loss: 2.4210
Epoch 46/50
125/125 ━━━━━━━━━━━━━━━━━━━━ 1s 7ms/step - loss: 2.2089 - val_loss: 2.4144
Epoch 47/50
125/125 ━━━━━━━━━━━━━━━━━━━━ 1s 8ms/step - loss: 2.2964 - val_loss: 2.4131
Epoch 48/50
125/125 ━━━━━━━━━━━━━━━━━━━━ 1s 8ms/step - loss: 2.2985 - val_loss: 2.4102
Epoch 50/50
125/125 ━━━━━━━━━━━━━━━━━━━━ 1s 8ms/step - loss: 2.2897 - val_loss: 2.4093
157/157 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - loss: 2.2898
125/125 ━━━━━━━━━━━━━━━━━━━━ 41s 13ms/step - loss: 0.3890 - val_loss: 0.4742
Epoch 2/50
125/125 ━━━━━━━━━━━━━━━━━━━━ 2s 12ms/step - loss: 0.4289 - val_loss: 0.3827
Epoch 3/50
125/125 ━━━━━━━━━━━━━━━━━━━━ 1s 10ms/step - loss: 0.3673 - val_loss: 0.3546
Epoch 4/50
125/125 ━━━━━━━━━━━━━━━━━━━━ 2s 9ms/step - loss: 0.3412 - val_loss: 0.3400
Epoch 5/50
125/125 ━━━━━━━━━━━━━━━━━━━━ 1s 9ms/step - loss: 0.3298 - val_loss: 0.3307
Epoch 6/50
125/125 ━━━━━━━━━━━━━━━━━━━━ 1s 9ms/step - loss: 0.3207 - val_loss: 0.3184
Epoch 7/50
125/125 ━━━━━━━━━━━━━━━━━━━━ 1s 7ms/step - loss: 0.3170 - val_loss: 0.3191
Epoch 8/50
125/125 ━━━━━━━━━━━━━━━━━━━━ 1s 8ms/step - loss: 0.3095 - val_loss: 0.3153
Epoch 9/50
125/125 ━━━━━━━━━━━━━━━━━━━━ 1s 8ms/step - loss: 0.3077 - val_loss: 0.3082
Epoch 10/50
125/125 ━━━━━━━━━━━━━━━━━━━━ 1s 8ms/step - loss: 0.3040 - val_loss: 0.3092
Epoch 11/50
125/125 ━━━━━━━━━━━━━━━━━━━━ 1s 10ms/step - loss: 0.3039 - val_loss: 0.3070
Epoch 12/50
125/125 ━━━━━━━━━━━━━━━━━━━━ 1s 8ms/step - loss: 0.3027 - val_loss: 0.3206
Epoch 13/50
125/125 ━━━━━━━━━━━━━━━━━━━━ 2s 11ms/step - loss: 0.3048 - val_loss: 0.3051
Epoch 14/50
125/125 ━━━━━━━━━━━━━━━━━━━━ 1s 8ms/step - loss: 0.2988 - val_loss: 0.3033
Epoch 15/50
125/125 ━━━━━━━━━━━━━━━━━━━━ 1s 11ms/step - loss: 0.3006 - val_loss: 0.3235
Epoch 16/50
125/125 ━━━━━━━━━━━━━━━━━━━━ 2s 11ms/step - loss: 0.3017 - val_loss: 0.3134
Epoch 17/50
125/125 ━━━━━━━━━━━━━━━━━━━━ 1s 8ms/step - loss: 0.3048 - val_loss: 0.3027
Epoch 18/50
125/125 ━━━━━━━━━━━━━━━━━━━━ 1s 7ms/step - loss: 0.2982 - val_loss: 0.3063
Epoch 19/50
125/125 ━━━━━━━━━━━━━━━━━━━━ 1s 8ms/step - loss: 0.2988 - val_loss: 0.3124
Epoch 20/50
125/125 ━━━━━━━━━━━━━━━━━━━━ 1s 5ms/step - loss: 0.2971 - val_loss: 0.3052
Epoch 21/50
125/125 ━━━━━━━━━━━━━━━━━━━━ 1s 8ms/step - loss: 0.2986 - val_loss: 0.3062
Epoch 22/50
125/125 ━━━━━━━━━━━━━━━━━━━━ 1s 8ms/step - loss: 0.2974 - val_loss: 0.3049
Epoch 23/50
125/125 ━━━━━━━━━━━━━━━━━━━━ 1s 8ms/step - loss: 0.2967 - val_loss: 0.3049
Epoch 24/50
125/125 ━━━━━━━━━━━━━━━━━━━━ 2s 13ms/step - loss: 0.2979 - val_loss: 0.3054
Epoch 25/50
125/125 ━━━━━━━━━━━━━━━━━━━━ 2s 8ms/step - loss: 0.3001 - val_loss: 0.3057
Epoch 26/50
125/125 ━━━━━━━━━━━━━━━━━━━━ 1s 11ms/step - loss: 0.2952 - val_loss: 0.3254
Epoch 27/50
125/125 ━━━━━━━━━━━━━━━━━━━━ 2s 8ms/step - loss: 0.2972 - val_loss: 0.3047
Epoch 28/50
125/125 ━━━━━━━━━━━━━━━━━━━━ 2s 10ms/step - loss: 0.2975 - val_loss: 0.3013
Epoch 29/50
125/125 ━━━━━━━━━━━━━━━━━━━━ 1s 12ms/step - loss: 0.2947 - val_loss: 0.3053
Epoch 30/50
125/125 ━━━━━━━━━━━━━━━━━━━━ 2s 8ms/step - loss: 0.2937 - val_loss: 0.3059
Epoch 31/50
125/125 ━━━━━━━━━━━━━━━━━━━━ 2s 12ms/step - loss: 0.2930 - val_loss: 0.3013
Epoch 32/50
125/125 ━━━━━━━━━━━━━━━━━━━━ 2s 12ms/step - loss: 0.2945 - val_loss: 0.3285
Epoch 33/50
125/125 ━━━━━━━━━━━━━━━━━━━━ 1s 5ms/step - loss: 0.2961 - val_loss: 0.3030
Epoch 34/50
125/125 ━━━━━━━━━━━━━━━━━━━━ 1s 8ms/step - loss: 0.2932 - val_loss: 0.3104
Epoch 35/50
125/125 ━━━━━━━━━━━━━━━━━━━━ 1s 5ms/step - loss: 0.2972 - val_loss: 0.3138
Epoch 36/50
125/125 ━━━━━━━━━━━━━━━━━━━━ 1s 8ms/step - loss: 0.2945 - val_loss: 0.3035
Epoch 37/50
125/125 ━━━━━━━━━━━━━━━━━━━━ 1s 8ms/step - loss: 0.2922 - val_loss: 0.3017
Epoch 38/50
125/125 ━━━━━━━━━━━━━━━━━━━━ 1s 8ms/step - loss: 0.2920 - val_loss: 0.2996
Epoch 39/50
125/125 ━━━━━━━━━━━━━━━━━━━━ 1s 8ms/step - loss: 0.2931 - val_loss: 0.2999
Epoch 40/50
125/125 ━━━━━━━━━━━━━━━━━━━━ 1s 5ms/step - loss: 0.2916 - val_loss: 0.3049
Epoch 41/50
125/125 ━━━━━━━━━━━━━━━━━━━━ 1s 5ms/step - loss: 0.2922 - val_loss: 0.2999
Epoch 42/50
125/125 ━━━━━━━━━━━━━━━━━━━━ 1s 8ms/step - loss: 0.2892 - val_loss: 0.2997
Epoch 43/50
125/125 ━━━━━━━━━━━━━━━━━━━━ 1s 8ms/step - loss: 0.2939 - val_loss: 0.2997
Epoch 44/50
125/125 ━━━━━━━━━━━━━━━━━━━━ 1s 8ms/step - loss: 0.2899 - val_loss: 0.2978
Epoch 45/50
125/125 ━━━━━━━━━━━━━━━━━━━━ 1s 8ms/step - loss: 0.2924 - val_loss: 0.3054
Epoch 46/50
125/125 ━━━━━━━━━━━━━━━━━━━━ 1s 8ms/step - loss: 0.2908 - val_loss: 0.3050
Epoch 47/50
125/125 ━━━━━━━━━━━━━━━━━━━━ 1s 8ms/step - loss: 0.2924 - val_loss: 0.2990
Epoch 48/50
125/125 ━━━━━━━━━━━━━━━━━━━━ 1s 8ms/step - loss: 0.2913 - val_loss: 0.2996
Epoch 49/50
125/125 ━━━━━━━━━━━━━━━━━━━━ 1s 8ms/step - loss: 0.2915 - val_loss: 0.2984
Epoch 50/50
125/125 ━━━━━━━━━━━━━━━━━━━━ 1s 8ms/step - loss: 0.2982 - val_loss: 0.3015
157/157 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.2989
```

- After training with this alternative loss, do the following:
- Compare the reconstruction MSE from this experiment with the MSE reported in Q2:

[In [28]]

```
# Compare MSEs
print("MSE (Experiment 2 MSE, Latent Size = 10):", mse_10)
print("MSE (Nonzero MSE, Latent Size = 10):", mse_nz)
print("The reconstruction MSE for this experiment is much less than the MSE from Experiment 2.")
```

```
MSE (Experiment 2 MSE, Latent Size = 10): 2.5382508476348091
MSE (Nonzero MSE, Latent Size = 10): 0.3015000000000001
The reconstruction MSE for this experiment is much less than the MSE from Experiment 2.
```

Generate the following four plots:

- PCA plot: Reconstructed data vs. Original data.
- PCA plot: Latent embeddings vs. Original data.
- t-SNE plot: Reconstructed data vs. Original data.
- t-SNE plot: Latent embeddings vs. Original data.

[In [29]]

```
# Get reconstructed data
X_reconstructed = autoencoder.predict(counts)
latent_embeddings = encoder.predict(counts)

# Compute PCA for Original vs. Reconstructed Data
pca = PCA(n_components=2)
X_original_pca = pca.fit_transform(counts)
X_reconstructed_pca = pca.transform(X_reconstructed)

plt.figure(figsize=(8,6))
plt.scatter(X_original_pca[:,0], X_original_pca[:,1], alpha=0.5, label="Original")
plt.scatter(X_reconstructed_pca[:,0], X_reconstructed_pca[:,1], alpha=0.5, label="Reconstructed")
plt.legend()
plt.title("PCA: Original vs. Reconstructed Data")
plt.show()

# PCA: Latent Embeddings vs. Original Data
X_latent_pca = pca.fit_transform(latent_embeddings)

plt.figure(figsize=(8,6))
plt.scatter(X_original_pca[:,0], X_original_pca[:,1], alpha=0.5, label="Original")
plt.scatter(X_latent_pca[:,0], X_latent_pca[:,1], alpha=0.5, label="Latent Embeddings")
plt.legend()
plt.title("PCA: Latent Embeddings vs. Original Data")
plt.show()

# Compute t-SNE for Original vs. Reconstructed Data
tsne = TSNE(n_components=2, random_state=42)
X_original_tsne = tsne.fit_transform(counts)
X_reconstructed_tsne = tsne.fit_transform(X_reconstructed)

plt.figure(figsize=(8,6))
plt.scatter(X_original_tsne[:,0], X_original_tsne[:,1], alpha=0.5, label="Original")
plt.scatter(X_reconstructed_tsne[:,0], X_reconstructed_tsne[:,1], alpha=0.5, label="Reconstructed")
plt.legend()
plt.title("t-SNE: Original vs. Reconstructed Data")
plt.show()

# t-SNE: Latent Embeddings vs. Original Data
X_latent_tsne = tsne.fit_transform(latent_embeddings)

plt.figure(figsize=(8,6))
plt.scatter(X_original_tsne[:,0], X_original_tsne[:,1], alpha=0.5, label="Original")
plt.scatter(X_latent_tsne[:,0], X_latent_tsne[:,1], alpha=0.5, label="Latent Embeddings")
plt.legend()
plt.title("t-SNE: Latent Embeddings vs. Original Data")
plt.show()
```

```
157/157 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step
157/157 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step
```

## Discuss:

- Discuss any observed differences, noting if the model with the new loss better captures the relevant structure in the data by focusing on nonzero values

PCA, Reconstructed Data vs. Original Data: The reconstructed cluster overlaps the original data structure, but the reconstructed cluster is more compact than the original data structure. Since the reconstructed data points overlap well with the original ones, the model has successfully retained important features.

PCA, Latent Embeddings vs. Original Data: The latent embeddings are clustered around a line, whereas the original data has the shape of a normal cluster.

t-SNE, Reconstructed Data vs. Original Data: The original data is clustered in a small, compact circle, whereas the reconstructed data is wide-spread. Since the original vs. reconstructed data plot does show overlapping clusters, it suggests that the autoencoder maintains meaningful relationships

t-SNE, Latent Embeddings vs. Original Data: The original data is clustered in a small, compact circle, whereas the reconstructed data appears in 5 separate clusters surrounding the original data. Since the clusters remain distinct, the model effectively captures meaningful structure. Since the plot still captures major clusters, the latent space is well-structured.

Effect of New (Nonzero MSE) Loss: Since this custom loss focuses on nonzero values, the reconstructed data better preserves active features compared to a standard MSE loss. Because the plots show that the clusters appear more distinct in the reconstructed or latent embeddings compared to standard autoencoders, the loss function is likely helping retain relevant structure. Thus, the model with the new loss better captures the relevant structure in the data by focusing on nonzero values.

[In [ ]]